C# ja .NET MAUI

Projekti programmeerimiskeel ja raamistik
Meie projekt kasutab C# keelt ning programm on loodud .NET MAUI raamistikule. C# kood töötab .NET runtime peal. Runtime käivitab programmi, haldab mälu, ning laseb samal programmil töötada nii Androidi kui ka Windowsi peal.
.NET MAUI raamistiku peamine eesmärk ongi võimaldada ühe projekti ja ühise koodibaasiga luua rakendusi mitmele platvormile. .NET MAUI kasutab binding'uid, mis seovad andmeid programmi loogika ja kasutajaliidese vahel.

Sarnasused ja erinevused teiste keeltega
C# programmeerimiskeel on sarnane mitme erineva laialt kasutatava keelega, näiteks C++ ja Java. Peamiselt on sarnasused C-laadse süntaksiga keeltega ning keeltega, mis võimaldavad objektorienteeritud programmeerimist.
Kõige suurem erinevus tuleneb C# kasutamisest koos .NET MAUI raamistikuga. Mobiilirakenduses tegeleb C# programmi loogikaga, kuid mobiilirakendusel tuleb ka arvestada kasutajaliidese oleku, asünkroonsete tegevustega jms, millega aitab .NET MAUI.

Keele tüübisüsteem
C# on tugevalt tüübitud keel, ehk muutujatel on kindel tüüp ning kompilaator kontrollib tüübiõigsust enne programmi käivitamist. C#-s on iga tüüp kas väärtustüüp või viitetüüp. Väärtustüübiga muutuja hoiab väärtust endas. Viitetüübiga muutuja hoiab viidet mingile objektile. C#-s kasutatakse puuduva väärtuse või viite tähistamiseks nulli ning nullable reference type, mis lubab märkida "?" abil, et muutuja väärtus võib olla 0.

Mälu- ja objektide elutsükli mudel
C# kasutab garbage collection-it, ehk erinevalt C++-st ei pea programmeerija ise mälu vabastama.
Kui luua uus objekt:
var thing = new Thing();
siis objekt paigutatakse mällu. Kui sellele objektile ei ole enam kasutatavaid viiteid, eemaldab garbage collector selle mälust.
Mobiilirakenduses on mälu rohkem piiratud, kui lauaarvutis, ehk vaatamata automaatsele mälu vabastamisele peab programmeerija nägema vaeva, et kasutamata objektid ei hoiaks kogemata viiteid.

Asünkroonsus ja konkurentsus
C# toetab asünkroonset programmeerimist peamiselt async ja await abil. Asünkroonsus võimaldab rakendusel alustada pikemat aega võtvat tegevust, näiteks andmete laadimist internetist või failist, ilma et kasutajaliides selle ajaks kinni jääks. Kui tegevus ootab näiteks serverilt vastust, saab rakendus samal ajal jätkata teiste

ülesannete täitmist. .NET MAUI mobiilirakenduses on see eriti oluline, sest kasutajaliides peab jääma reageerivaks. Näiteks kui rakendus peab laadima andmeid internetist, saab kasutada await, et oodata vastuse saabumist ilma kasutajaliidese blokeerimiseta.

```
async Task LoadData()
{
   var data = await GetDataFromServerAsync();
}
```

Siin näitab async, et meetod võib töötada asünkroonselt, ning await võimaldab oodata GetDataFromServerAsync() tulemuse saabumist ilma kogu rakenduse töö peatamiseta.

Kuidas jõuab kood mobiiliplatvormi API-deni

C# kood jõuab mobiiliplatvormi API-deni .NET MAUI kaudu. See tähendab, et on võimalik kirjutada C# koodi ning .NET MAUI suhtleb selle kaudu Androidi või iOS-i süsteemiga. Näiteks telefoni GPS-i kasutamiseks ei pea Androidi ja iOS-i jaoks eraldi koodi kirjutama. Kui mõnda funktsiooni .NET MAUI ei toeta, saab selle jaoks kirjutada eraldi Androidi või iOS-i koodi.
.NET MAUI abstraktsioonikiht: C# kood -> .NET MAUI -> platvormi API -> operatsioonisüsteem -> riistvara.

.NET MAUI arhitektuur

.NET MAUI puhul on võimalik kasutada MVVM (Model-View-ViewModel) arhitektuurimustrit. See eraldab kasutajaliidese, andmed ja kasutajaliidese loogika.
Model sisaldab andmeid ja nende struktuuri.
View on kasutajale nähtav kasutajaliides.
ViewModel vahendab andmeid ja sisaldab kasutajaliidese loogikat.
Selle juures on oluline data binding ehk andmete sidumine.
Näiteks võib XAML-is olla:

```
<Entry Text="{Binding UserName}" />
```

Sellisel juhul ei pea C#-is iga kord käsitsi kasutajaliidese tekstivälja väärtust muutma. Kasutajaliides on seotud ViewModel-is oleva omadusega.

Allikad

https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui?view=net-maui-10.0
https://learn.microsoft.com/en-us/dotnet/maui/user-interface/handlers/?view=net-maui-10.0
https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm
https://learn.microsoft.com/en-us/dotnet/standard/garbage-collection/fundamentals
https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/
https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/